Marwadi University
Marwadi Chandarana Group

NAAC A+

Exception Handling:

What is exception:

An exception in java programming is an abnormal situation that is raised during the program execution. In simple words, an exception is a problem that arises at the time of program execution.

Reasons for Exception Occurrence:

Several reasons lead to the occurrence of an exception. A few of them are as follows.

- When we try to open a file that does not exist may lead to an exception.
- When the user enters invalid input data, it may lead to an exception.
- When a network connection has lost during the program execution may lead to an exception.
- When we try to access the memory beyond the allocated range may lead to an exception.

Exception Handling:

The **exception handling** is the mechanism to handle the runtime errors to maintain the normal flow of the application.
Exception normally disrupts the normal flow of the application that is why we use exception handling.

Cont..

Fundamentals of exception handling: -

In java, the exception handling mechanism uses five keywords namely try, catch, finally, throw, and throws.
The keyword **try** is used to define a block of code that will be tests the occurence of an exception.

The keyword **catch** is used to define a block of code that handles the exception occured in the respective try block.

The **finally** block always executes, whether an exception occurs or not, and is used for cleanup tasks like closing files or connections.

The **throw** keyword in Java is used to **manually trigger an exception**.

The **throws** keyword specifies the exceptions that a method can throw to the default handler and does not handle itself. That means when we need a method to throw an exception automatically, we use throws keyword followed by method declaration.

The general form of an exception handling block


```
try{
// block of code to monitor for errors
}
Catch(ExceptionType1 exOb){
//exception handler for Exception type1
}
Catch(ExceptionType2 exOb){
//exception handler for Exception type2
}
finally{
// block of code to be executed after try block ends
}
```

Types of Exceptions

In java, exceptions have categorized into two types, and they are as follows.

Checked Exceptions- An exception that is checked by the compiler at the time of compilation are called checked exceptions.

Unchecked Exceptions - An exception that cannot be caught by the compiler but occurs at the time of program executions are called unchecked exceptions.

Uncaught Exceptions in Java:-

The uncaught exceptions are the exceptions that are not caught by the compiler but automatically caught and handled by the Java built-in exception handler.

The JVM's default exception handler prints the error message and terminates the thread.

Java provides a strong exception handling mechanism using keywords like **try, catch, finally, throw, and throws** to handle such situations.

Ex:-

```
class UncaughtExceptionExample {
    public static void main(String args[]) {
        int x = 15, y = 0, z;
        z = x / y;
        System.out.println(z);
    }
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at UncaughtExceptionExample.main(UncaughtExceptionExample.java:4)
```

Handle the Exception using Try ,Catch and Finally:

```
class ExceptionExample {
   public static void main(String args[]) {
      int x = 15, y = 0, z;
      try {
         z = x / y;
         System.out.println(z);
      }
      catch (ArithmeticException ae) {
         System.out.println(ae);
      }
      finally {
         System.out.println("finally block is executed always");
      }
   }
}
```

Output:-

java.lang.ArithmeticException: / by zero
finally block is executed always

Multiple catch clauses

In java programming language, a try block may has one or more number of catch blocks. That means a single try statement can have multiple catch clauses.

When a try block has more than one catch block, each catch block must contain a different exception type to be handled.

Ex:-

```
public class MultipleCatch {
   public static void main(String[] args) {
      try {
         int a[] = new int[5];
         a[5] = 30 / 0;
      }
      catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception occurs");
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("ArrayIndexOutOfBounds Exception occurs");
      }
      catch (Exception e) {
         System.out.println("Parent Exception occurs");
      }
      System.out.println("Remaining code gets executed");
   }
}
```

Output:-

Arithmetic Exception occurs
Remaining code gets executed

Ex:-

```
public class MultipleCatch {
   public static void main(String[] args) {
      try {
         int a[] = new int[5];
         a[5] = 30 / 0;
      }
      catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception occurs");
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("ArrayIndexOutOfBounds Exception occurs");
      }
      catch (Exception e) {
         System.out.println("Parent Exception occurs");
      }
      System.out.println("Remaining code gets executed");
   }
}
```

Output:-

Arithmetic Exception occurs
Remaining code gets executed

What is a Stream

A **stream** is a sequence (flow) of data

Streams are divided into two types:

- 1. Byte Streams
- 2. Character Streams

1. Byte Streams

- Used to handle **raw binary data** (e.g., images, audio, files).
- **Superclass:** InputStream and OutputStream

FileInputStream

- Reads **raw bytes** from a file.
- Best for reading binary files like images or videos.

Key Methods:

int read() // reads one byte
int read(byte[] b) // reads multiple bytes
void close()

Example:-

```
FileInputStream fis = new FileInputStream("file.txt");
int data = fis.read();
while(data != -1) {
System.out.print((char) data);
data = fis.read();
}
fis.close();
```

Cont..

FileOutputStream

- Writes **raw bytes** to a file.
- Ideal for writing binary content.

Key Methods:

void write(int b) // writes one byte
void write(byte[] b) // writes byte array
void close()

Example:

```
FileOutputStream fos = new FileOutputStream("file.txt");
fos.write("Hello World".getBytes());
fos.close();
```

2. Character Streams

- 2. Character Streams

  - Used to handle **text data (characters)**.
  - **Superclass:** Reader and Writer

Reader (Abstract Class)	Writer (Abstract Class)
Base class for all character input streams.	Base class for all character output streams.
Common Methods: int read() int read(char[] cbuf) void close()	**Common Methods:** void write(int c) void write(char[] cbuf) void write(String str) void close()

Input Stream Reader

- **Bridges byte streams to character streams**.
- Converts bytes from an InputStream into characters using a character encoding.

Constructor:

InputStreamReader(InputStream in)
InputStreamReader(InputStream in, String charsetName)

Example:

```
FileInputStream fis = new FileInputStream("file.txt");
InputStreamReader isr = new InputStreamReader(fis);
int ch = isr.read();
while(ch != -1) {
System.out.print((char) ch);
ch = isr.read();
}
isr.close();
```

Output Stream Writer class

- **Bridges character streams to byte streams**.
- Converts characters into bytes using a specified encoding.

Constructor:

OutputStreamWriter(OutputStream out)
OutputStreamWriter(OutputStream out, String charsetName)

Example:

```
FileOutputStream fos = new FileOutputStream("file.txt");
OutputStreamWriter osw = new OutputStreamWriter(fos);
osw.write("Hello Java");
osw.close();
```

BufferedReader and BufferedWriter

BufferedReader and BufferedWriter make reading and writing **text files** faster.

BufferedReader lets you read text **line by line** with readLine().

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class BufferedReaderExample {
    public static void main(String[] args) {
        // Use try-with-resources to ensure the reader is closed automatically
        try (BufferedReader br = new BufferedReader(new FileReader("example.txt"))) {
            String line;
            // The readLine() method reads a line of text at a time
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

BufferedReader and BufferedWriter

BufferedWriter lets you write text efficiently and add new lines with newLine().

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BufferedWriterExample {
    public static void main(String[] args) {
        String[] lines = {"Line one", "Line two", "Line three"};

        // Use try-with-resources to ensure the writer is closed automatically
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {
            for (String line : lines) {
                bw.write(line);
                bw.newLine(); // Writes a line separator
            }
        } catch (IOException e) {
            System.out.println("Error writing to file: " + e.getMessage());
        }
    }
}
```

THANK YOU